

Training Future Software Developers to Acquire Agile Development Skills

Giving students experience developing and delivering large-scale systems under time constraints and shifting deadlines.

Agile techniques are fast becoming the adopted development methodology commercially. Central to the approach is that a project team can be effective in responding to the changing environment and user requirements if it capitalizes on its members' skills, stresses collaboration among members and users, emphasizes functional modularity, mandates short iteration cycles to enable fast verifications of user requirements, removes all unnecessary activities (such as redundant documentation), provides more system releases with smaller changes between each release, and focuses on developing functions up to the point of adequacy. This column reports the experience of imparting agile techniques to students by guiding them toward delivering large-scale systems.

Information Systems Development Project (ISDP) is an advanced course offered to students in their third year of study who have satisfied prerequisites that are related to software engi-

Agile Principles	Actions
Planning and dynamic prioritizing of functions to deliver.	Demand prioritized function list.
Functional modularity.	Use of multitiered software development platform (such as the Java 2 Enterprise Edition (J2EE) platform ¹).
Early and continuous delivery of software.	Mandate multiple system releases.
Accommodate changing requirements at any time.	Release second set of requirements toward second half of semester.
Constant feedback and change.	Assign an advisor (acting as user) to each team. Conduct debriefing sessions after each release.
Stress on collaboration.	Deliver lectures on project management and conflict resolution.
Environment to motivate developers.	Provide students with development software and computer facilities. Advisors send email recognizing their efforts regularly.
Working software is the primary assessment measure.	Main assessment is on software delivered.
Continuous attention to good design and technical excellence.	Deliver lectures on good system design principles, and conduct specialized workshops on enterprise-level programming. Advisors review proposed system architectures.
Emphasis on simplicity.	Constantly caution students that assessment is on system as a whole rather than on individual function.
Teams reflect on how to become more effective at regular intervals.	Demand peer reviews on development progress and other members' performances at regular intervals. Adjust supervision based on peer review outcomes and observed progress.
Promote sustainable development.	Mandate regular meetings with advisors. Define specific deadlines for system releases.
Maintain close working relationship with users.	Teams meet with their advisors weekly to clarify requirements and doubts and report their progress.

¹<http://java.sun.com/j2ee/>

Key course design.

neering, database systems, operating systems, and Java programming. Through ISDP, students experience, for the first time, a simulated but realistic software

development environment that puts sufficiently high time pressure on them to deliver the system and injects the element of uncertainty in functional requirements.

The agile principles incorporated into the course structure are

listed in the table here. Students form teams of five or six members each. Several milestones and system releases are stipulated to provide students with the opportunity to learn to pay attention to technical excellence and design agility and to review and revise system deliveries. An element of “shock” that disrupts system development is explicitly introduced: Students are not informed of the second set of functional requirements until the ninth week. This is to develop their ability to respond to the ever-changing real-world system requirements. Expecting the students to deliver working systems within 13 weeks also forces them to only code sufficiently for each function (the principle of simplicity).

We compare the difference in system quality delivered before and after the agile techniques were adopted in the course. We observe, for two consecutive semesters, the use of agile techniques has positive impact on development progress and system quality based on students’ feedback and external examiners’ evaluations. Ten additional recommendations are derived, and are described in more detail here:

Mandate function list: This provides students with an introduction to project planning and system requirement documentation. The planning process encourages students to think about issues related to their capacity in delivering system functions and meeting user priorities.

Be pragmatic: Encourage students to develop a system to the point of adequacy rather than excessiveness to cater to future uncertain needs. Releasing detailed system evaluation criteria breakdown early (for example, user interface, breadth and depth of system delivered, and so forth) induces students to be more inclined to “satisfice” on individual functions and focus more on delivering a working system as a whole.

Conceptualize unit testing before coding: Developing unit testing cases early and prior to code implementation could ferret out misconceptualization and misinterpretation in system requirements.

Multitiered software development platform: With a multitiered software development platform, such as J2EE, students are able to develop the system modularly and isolate bugs in specific tiers.

Continuous integration of the system: Demand functions to be integrated for each system release. Students are coached to integrate system components and perform integration testing for each release.

Demand multiple releases: Students are provided with an opportunity to reflect on the system just released and identify areas for improvement. Such reflection allows them to identify flaws in system design and function delivered early.

Introduce “shock”: Additional functions or requirement changes could be introduced. In our experience, to effectively introduce “shocks,” you need to first cus-

tomize the degree of changes to be made and additional function complexity according to students’ programming ability. Second, “shocks” could be framed as “challenges” to minimize resentments, and advisors should encourage students to incorporate the changes into their systems.

Teach conflict resolution: We observe many teams have a tendency to divide the work without much knowledge of the assigned members’ capabilities. When they could not deliver, conflict arises. Conflict management should be delivered in the lecture.

Perform post-mortem audit after each release: Main goals are to find the core causes for software flaws and provide recommendations so similar flaws do not occur in future release(s). Opportunities for improvement should also be identified.

Reflect on members’ roles and responsibilities: Each team member needs to understand his or her role in the project. Understanding strengths and identifying ways to address weaknesses of each member could be beneficial to the distribution of work. We recommend having a peer review session after each system release to highlight any team conflict early. **C**

CHUAN-HOO TAN (tanch@comp.nus.edu.sg) is an instructor in the Department of Information Systems, National University of Singapore, Singapore.

HOCK-HAI TEO (teohhh@comp.nus.edu.sg) is an associate professor in the Department of Information Systems, National University of Singapore, Singapore.
